

Map-Reduce Examples

NOTE

Aggregation Pipeline as Alternative to Map-Reduce

An aggregation pipeline provides better performance and usability than a map-reduce operation.

Map-reduce operations can be rewritten using aggregation pipeline stages such as `$group` and `$merge`.

For map-reduce operations that require custom functionality, MongoDB provides the `$accumulator` and `$function` aggregation operators. Use these operators to define custom aggregation expressions in JavaScript.

In `mongosh`, the `db.collection.mapReduce()` method is a wrapper around the `mapReduce` command. The following examples use the `db.collection.mapReduce()` method.

The examples in this section include aggregation pipeline alternatives without custom aggregation expressions. For alternatives that use custom expressions, see Map-Reduce to Aggregation Pipeline Translation Examples.

Create a sample collection `orders` with these documents:

```
db.orders.insertMany([
  { _id: 1, cust_id: "Ant O. Knee", ord_date: new Date("2020-03-01"),
    price: 25, items: [ { sku: "oranges", qty: 5, price: 2.5 }, { sku:
    "apples", qty: 5, price: 2.5 } ], status: "A" },
  { _id: 2, cust_id: "Ant O. Knee", ord_date: new Date("2020-03-08"),
    price: 70, items: [ { sku: "oranges", qty: 8, price: 2.5 }, { sku:
    "chocolates", qty: 5, price: 10 } ], status: "A" },
  { _id: 3, cust_id: "Busby Bee", ord_date: new Date("2020-03-08"), price:
  50, items: [ { sku: "oranges", qty: 10, price: 2.5 }, { sku: "pears",
  qty: 10, price: 2.5 } ], status: "A" },
  { _id: 4, cust_id: "Busby Bee", ord_date: new Date("2020-03-18"), price:
  25, items: [ { sku: "oranges", qty: 10, price: 2.5 } ], status: "A" },
  { _id: 5, cust_id: "Busby Bee", ord_date: new Date("2020-03-19"), price:
  50, items: [ { sku: "chocolates", qty: 5, price: 10 } ], status: "A"},
  { _id: 6, cust_id: "Cam Elot", ord_date: new Date("2020-03-19"), price:
  35, items: [ { sku: "carrots", qty: 10, price: 1.0 }, { sku: "apples",
  qty: 10, price: 2.5 } ], status: "A" },
  { _id: 7, cust_id: "Cam Elot", ord_date: new Date("2020-03-20"), price:
  25, items: [ { sku: "oranges", qty: 10, price: 2.5 } ], status: "A" },
  { _id: 8, cust_id: "Don Quis", ord_date: new Date("2020-03-20"), price:
  75, items: [ { sku: "chocolates", qty: 5, price: 10 }, { sku: "apples",
  qty: 10, price: 2.5 } ], status: "A" },
  { _id: 9, cust_id: "Don Quis", ord_date: new Date("2020-03-20"), price:
  55, items: [ { sku: "carrots", qty: 5, price: 1.0 }, { sku: "apples",
  qty: 10, price: 2.5 }, { sku: "oranges", qty: 10, price: 2.5 } ],
  status: "A" },
  { _id: 10, cust_id: "Don Quis", ord_date: new Date("2020-03-23"), price:
  25, items: [ { sku: "oranges", qty: 10, price: 2.5 } ], status: "A" }
])
```



Return the Total Price Per Customer

Perform the map-reduce operation on the `orders` collection to group by the `cust_id`, and calculate the sum of the `price` for each `cust_id`:

1. Define the map function to process each input document:

- In the function, `this` refers to the document that the map-reduce operation is processing.
- The function maps the `price` to the `cust_id` for each document and emits the `cust_id` and `price`.

```
var mapFunction1 = function() {  
  emit(this.cust_id, this.price);  
};
```

2. Define the corresponding reduce function with two arguments `keyCustId` and `valuesPrices`:

- The `valuesPrices` is an array whose elements are the `price` values emitted by the map function and grouped by `keyCustId`.
- The function reduces the `valuesPrice` array to the sum of its elements.

```
var reduceFunction1 = function(keyCustId, valuesPrices) {  
  return Array.sum(valuesPrices);  
};
```

3. Perform map-reduce on all documents in the `orders` collection using the `mapFunction1` map function and the `reduceFunction1` reduce function:

```
db.orders.mapReduce(  
  mapFunction1,  
  reduceFunction1,  
  { out: "map_reduce_example" }  
)
```

This operation outputs the results to a collection named `map_reduce_example`. If the `map_reduce_example` collection already exists, the operation will replace the contents with the results of this map-reduce operation.

4. Query the `map_reduce_example` collection to verify the results:

```
db.map_reduce_example.find().sort( { _id: 1 } )
```

The operation returns these documents:

```
{ "_id" : "Ant 0. Knee", "value" : 95 }  
{ "_id" : "Busby Bee", "value" : 125 }
```

```
{ "_id" : "Cam Elot", "value" : 60 }
{ "_id" : "Don Quis", "value" : 155 }
```

Aggregation Alternative

Using the available aggregation pipeline operators, you can rewrite the map-reduce operation without defining custom functions:

```
db.orders.aggregate([
  { $group: { _id: "$cust_id", value: { $sum: "$price" } } },
  { $out: "agg_alternative_1" }
])
```

1. The `$group` stage groups by the `cust_id` and calculates the `value` field (See also `$sum`). The `value` field contains the total `price` for each `cust_id`.

The stage output the following documents to the next stage:

```
{ "_id" : "Don Quis", "value" : 155 }
{ "_id" : "Ant O. Knee", "value" : 95 }
{ "_id" : "Cam Elot", "value" : 60 }
{ "_id" : "Busby Bee", "value" : 125 }
```

2. Then, the `$out` writes the output to the collection `agg_alternative_1`. Alternatively, you could use `$merge` instead of `$out`.
3. Query the `agg_alternative_1` collection to verify the results:

```
db.agg_alternative_1.find().sort( { _id: 1 } )
```

The operation returns the following documents:

```
{ "_id" : "Ant O. Knee", "value" : 95 }
{ "_id" : "Busby Bee", "value" : 125 }
{ "_id" : "Cam Elot", "value" : 60 }
{ "_id" : "Don Quis", "value" : 155 }
```

TIP

See also:

For an alternative that uses custom aggregation expressions, see [Map-Reduce to Aggregation Pipeline Translation Examples](#).

Calculate Order and Total Quantity with Average Quantity Per Item

In the following example, you will see a map-reduce operation on the `orders` collection for all documents that have an `ord_date` value greater than or equal to `2020-03-01`.

The operation in the example:

1. Groups by the `item.sku` field, and calculates the number of orders and the total quantity ordered for each `sku`.
2. Calculates the average quantity per order for each `sku` value and merges the results into the output collection.

When merging results, if an existing document has the same key as the new result, the operation overwrites the existing document. If there is no existing document with the same key, the operation inserts the document.

Example steps:

1. Define the map function to process each input document:
 - In the function, `this` refers to the document that the map-reduce operation is processing.
 - For each item, the function associates the `sku` with a new object `value` that contains the `count` of 1 and the item `qty` for the order and emits the `sku` (stored in the `key`) and the `value`.

```
var mapFunction2 = function() {  
  for (var idx = 0; idx < this.items.length; idx++) {  
    var key = this.items[idx].sku;  
    var value = { count: 1, qty: this.items[idx].qty };  
    emit(key, value);  
  }  
};
```

2. Define the corresponding reduce function with two arguments `keySKU` and `countObjVals`:
 - `countObjVals` is an array whose elements are the objects mapped to the grouped `keySKU` values passed by map function to the reducer function.
 - The function reduces the `countObjVals` array to a single object `reducedValue` that contains the `count` and the `qty` fields.
 - In `reducedVal`, the `count` field contains the sum of the `count` fields from the individual array elements, and the `qty` field contains the sum of the `qty` fields from the individual array elements.

```
var reduceFunction2 = function(keySKU, countObjVals) {  
  reducedVal = { count: 0, qty: 0 };  
  for (var idx = 0; idx < countObjVals.length; idx++) {  
    reducedVal.count += countObjVals[idx].count;  
    reducedVal.qty += countObjVals[idx].qty;  
  }  
  return reducedVal;  
};
```

3. Define a finalize function with two arguments `key` and `reducedVal`. The function modifies the `reducedVal` object to add a computed field named `avg` and returns the modified object:

```
var finalizeFunction2 = function (key, reducedVal) {  
  reducedVal.avg = reducedVal.qty/reducedVal.count;  
  return reducedVal;  
};
```

4. Perform the map-reduce operation on the `orders` collection using the `mapFunction2`, `reduceFunction2`, and `finalizeFunction2` functions:

```
db.orders.mapReduce(  
  mapFunction2,  
  reduceFunction2,  
  {  
    out: { merge: "map_reduce_example2" },  
    query: { ord_date: { $gte: new Date("2020-03-01") } },  
    finalize: finalizeFunction2  
  }  
);
```

This operation uses the `query` field to select only those documents with `ord_date` greater than or equal to `new Date("2020-03-01")`. Then it outputs the results to a collection `map_reduce_example2`.

If the `map_reduce_example2` collection already exists, the operation will merge the existing contents with the results of this map-reduce operation. That is, if an existing document has the same key as the new result, the operation overwrites the existing document. If there is no existing document with the same key, the operation inserts the document.

5. Query the `map_reduce_example2` collection to verify the results:

```
db.map_reduce_example2.find().sort( { _id: 1 } )
```

The operation returns these documents:

```
{ "_id" : "apples", "value" : { "count" : 4, "qty" : 35, "avg" : 8.75 }  
}  
{ "_id" : "carrots", "value" : { "count" : 2, "qty" : 15, "avg" : 7.5 }  
}  
{ "_id" : "chocolates", "value" : { "count" : 3, "qty" : 15, "avg" : 5 }  
}  
{ "_id" : "oranges", "value" : { "count" : 7, "qty" : 63, "avg" : 9 } }  
{ "_id" : "pears", "value" : { "count" : 1, "qty" : 10, "avg" : 10 } }
```

Using the available aggregation pipeline operators, you can rewrite the map-reduce operation without defining custom functions:

```
db.orders.aggregate( [
  { $match: { ord_date: { $gte: new Date("2020-03-01") } } },
  { $unwind: "$items" },
  { $group: { _id: "$items.sku", qty: { $sum: "$items.qty" }, orders_ids:
    { $addToSet: "$_id" } } },
  { $project: { value: { count: { $size: "$orders_ids" }, qty: "$qty",
    avg: { $divide: [ "$qty", { $size: "$orders_ids" } ] } } } },
  { $merge: { into: "agg_alternative_3", on: "_id", whenMatched:
    "replace", whenNotMatched: "insert" } }
] )
```

1. The `$match` stage selects only those documents with `ord_date` greater than or equal to `new Date("2020-03-01")`.
2. The `$unwind` stage breaks down the document by the `items` array field to output a document for each array element. For example:

```
{ "_id" : 1, "cust_id" : "Ant O. Knee", "ord_date" : ISODate("2020-03-01T00:00:00Z"), "price" : 25, "items" : { "sku" : "oranges", "qty" : 5, "price" : 2.5 }, "status" : "A" }
{ "_id" : 1, "cust_id" : "Ant O. Knee", "ord_date" : ISODate("2020-03-01T00:00:00Z"), "price" : 25, "items" : { "sku" : "apples", "qty" : 5, "price" : 2.5 }, "status" : "A" }
{ "_id" : 2, "cust_id" : "Ant O. Knee", "ord_date" : ISODate("2020-03-08T00:00:00Z"), "price" : 70, "items" : { "sku" : "oranges", "qty" : 8, "price" : 2.5 }, "status" : "A" }
{ "_id" : 2, "cust_id" : "Ant O. Knee", "ord_date" : ISODate("2020-03-08T00:00:00Z"), "price" : 70, "items" : { "sku" : "chocolates", "qty" : 5, "price" : 10 }, "status" : "A" }
{ "_id" : 3, "cust_id" : "Busby Bee", "ord_date" : ISODate("2020-03-08T00:00:00Z"), "price" : 50, "items" : { "sku" : "oranges", "qty" : 10, "price" : 2.5 }, "status" : "A" }
{ "_id" : 3, "cust_id" : "Busby Bee", "ord_date" : ISODate("2020-03-08T00:00:00Z"), "price" : 50, "items" : { "sku" : "pears", "qty" : 10, "price" : 2.5 }, "status" : "A" }
{ "_id" : 4, "cust_id" : "Busby Bee", "ord_date" : ISODate("2020-03-18T00:00:00Z"), "price" : 25, "items" : { "sku" : "oranges", "qty" : 10, "price" : 2.5 }, "status" : "A" }
{ "_id" : 5, "cust_id" : "Busby Bee", "ord_date" : ISODate("2020-03-19T00:00:00Z"), "price" : 50, "items" : { "sku" : "chocolates", "qty" : 5, "price" : 10 }, "status" : "A" }
...
```

3. The `$group` stage groups by the `items.sku`, calculating for each sku:
 - The `qty` field. The `qty` field contains the total `qty` ordered per each `items.sku` (See `$sum`).
 - The `orders_ids` array. The `orders_ids` field contains an array of distinct order `_id`'s for the `items.sku` (See `$addToSet`).

```
{ "_id" : "chocolates", "qty" : 15, "orders_ids" : [ 2, 5, 8 ] }
{ "_id" : "oranges", "qty" : 63, "orders_ids" : [ 4, 7, 3, 2, 9, 1, 10 ] }
{ "_id" : "carrots", "qty" : 15, "orders_ids" : [ 6, 9 ] }
{ "_id" : "apples", "qty" : 35, "orders_ids" : [ 9, 8, 1, 6 ] }
{ "_id" : "pears", "qty" : 10, "orders_ids" : [ 3 ] }
```

4. The `$project` stage reshapes the output document to mirror the map-reduce's output to have two fields `_id` and `value`. The `$project` sets:
5. The `$unwind` stage breaks down the document by the `items` array field to output a document for each array element. For example:

```
{ "_id" : 1, "cust_id" : "Ant O. Knee", "ord_date" : ISODate("2020-03-01T00:00:00Z"), "price" : 25, "items" : { "sku" : "oranges", "qty" : 5, "price" : 2.5 }, "status" : "A" }
{ "_id" : 1, "cust_id" : "Ant O. Knee", "ord_date" : ISODate("2020-03-01T00:00:00Z"), "price" : 25, "items" : { "sku" : "apples", "qty" : 5, "price" : 2.5 }, "status" : "A" }
{ "_id" : 2, "cust_id" : "Ant O. Knee", "ord_date" : ISODate("2020-03-08T00:00:00Z"), "price" : 70, "items" : { "sku" : "oranges", "qty" : 8, "price" : 2.5 }, "status" : "A" }
{ "_id" : 2, "cust_id" : "Ant O. Knee", "ord_date" : ISODate("2020-03-08T00:00:00Z"), "price" : 70, "items" : { "sku" : "chocolates", "qty" : 5, "price" : 10 }, "status" : "A" }
{ "_id" : 3, "cust_id" : "Busby Bee", "ord_date" : ISODate("2020-03-08T00:00:00Z"), "price" : 50, "items" : { "sku" : "oranges", "qty" : 10, "price" : 2.5 }, "status" : "A" }
{ "_id" : 3, "cust_id" : "Busby Bee", "ord_date" : ISODate("2020-03-08T00:00:00Z"), "price" : 50, "items" : { "sku" : "pears", "qty" : 10, "price" : 2.5 }, "status" : "A" }
{ "_id" : 4, "cust_id" : "Busby Bee", "ord_date" : ISODate("2020-03-18T00:00:00Z"), "price" : 25, "items" : { "sku" : "oranges", "qty" : 10, "price" : 2.5 }, "status" : "A" }
{ "_id" : 5, "cust_id" : "Busby Bee", "ord_date" : ISODate("2020-03-19T00:00:00Z"), "price" : 50, "items" : { "sku" : "chocolates", "qty" : 5, "price" : 10 }, "status" : "A" }
...
```

6. The `$group` stage groups by the `items.sku`, calculating for each sku:
 - The `qty` field. The `qty` field contains the total `qty` ordered per each `items.sku` using `$sum`.
 - The `orders_ids` array. The `orders_ids` field contains an array of distinct order `_id`'s for the `items.sku` using `$addToSet`.

```
{ "_id" : "chocolates", "qty" : 15, "orders_ids" : [ 2, 5, 8 ] }
{ "_id" : "oranges", "qty" : 63, "orders_ids" : [ 4, 7, 3, 2, 9, 1, 10 ] }
{ "_id" : "carrots", "qty" : 15, "orders_ids" : [ 6, 9 ] }
{ "_id" : "apples", "qty" : 35, "orders_ids" : [ 9, 8, 1, 6 ] }
{ "_id" : "pears", "qty" : 10, "orders_ids" : [ 3 ] }
```

7. The `$project` stage reshapes the output document to mirror the map-reduce's output to have two fields `_id` and `value`. The `$project` sets:

- the `value.count` to the size of the `orders_ids` array using `$size`.
- the `value.qty` to the `qty` field of input document.
- the `value.avg` to the average number of `qty` per order using `$divide` and `$size`.

```
{ "_id" : "apples", "value" : { "count" : 4, "qty" : 35, "avg" : 8.75 } }
{ "_id" : "pears", "value" : { "count" : 1, "qty" : 10, "avg" : 10 } }
{ "_id" : "chocolates", "value" : { "count" : 3, "qty" : 15, "avg" : 5 } }
{ "_id" : "oranges", "value" : { "count" : 7, "qty" : 63, "avg" : 9 } }
{ "_id" : "carrots", "value" : { "count" : 2, "qty" : 15, "avg" : 7.5 } }
```

8. Finally, the `$merge` writes the output to the collection `agg_alternative_3`. If an existing document has the same key `_id` as the new result, the operation overwrites the existing document. If there is no existing document with the same key, the operation inserts the document.

9. Query the `agg_alternative_3` collection to verify the results:

```
db.agg_alternative_3.find().sort( { _id: 1 } )
```



The operation returns the following documents:

```
{ "_id" : "apples", "value" : { "count" : 4, "qty" : 35, "avg" : 8.75 } }
{ "_id" : "carrots", "value" : { "count" : 2, "qty" : 15, "avg" : 7.5 } }
{ "_id" : "chocolates", "value" : { "count" : 3, "qty" : 15, "avg" : 5 } }
{ "_id" : "oranges", "value" : { "count" : 7, "qty" : 63, "avg" : 9 } }
{ "_id" : "pears", "value" : { "count" : 1, "qty" : 10, "avg" : 10 } }
```

TIP

See also:

For an alternative that uses custom aggregation expressions, see [Map-Reduce to Aggregation Pipeline Translation Examples](#).

TIP

See also:

- [Perform Incremental Map-Reduce](#)
- [Map-Reduce to Aggregation Pipeline](#)